

ZUM Semestrální práce - MAPF

Filip Niedoba

Květen 2026

Multi-Agent Path Finding je problém hledání bezkolizních cest pro několik agentů se stanoveným počátkem a cílem, tak aby délky cest agentů byly do součtu minimální. Tato semestrální práce řeší variantu "Disappearing agents", která předpokládá zmizení agenta po dosažení jeho cíle.

Práce obsahuje:

- formulaci MAPF nad mřížkovým grafem s povoleným čekáním,
- implementaci algoritmu **Conflict-Based Search (CBS)** se zaručenou optimalitou vůči sum-of-cost,
- low-level plánovač jako **A*** nad space-time grafem s předpočítanou BFS heuristikou a horní mezí na časový horizont,
- experimentální vyhodnocení na 6 mapách z benchmarku movingai.com při různém počtu agentů,
- porovnání úspěšnosti a doby řešení podle typu mapy (*random*, *room*, *maze*) a její velikosti (32×32 , 64×64 , 128×128).
- vizualizace průchodu v terminálu

Popis problému

Vstup

- Mřížkový graf V s překážkami. Buňky jsou buď průchozí (.) nebo překážkou (@).
- k agentů resp. souřadnice jejich počátku s_i a cíle g_i .

Výstup

Plán $\pi = (\pi_1, \dots, \pi_k)$, kde každé $\pi_i = (v_i^0, v_i^1, \dots)$ je posloupnost vrcholů taková, že:

- $v_i^0 = s_i$ a v nějakém čase T_i platí $v_i^{T_i} = g_i$,
- každý přechod $v_i^t \rightarrow v_i^{t+1}$ je pohyb na sousední vrchol nebo čekání (5 možných akcí celkem),
- **vertex conflict**: pro $i \neq j$ a t platí $v_i^t \neq v_j^t$,
- **edge conflict**: pro $i \neq j$ a t neplatí současně $v_i^t = v_j^{t+1}$ a $v_i^{t+1} = v_j^t$ (žádná výměna přes hranu).

Algoritmus a implementace

Byl použit **Conflict-Based Search (CBS)** se zaručenou optimalitou v jazyce C++ (cca 600 řádků).

Stručný popis algoritmu

1. Pomocí BFS spočítáme pro každého agenta zvlášť mapu vzdáleností do cíle z každého dostupného vrcholu. To bude sloužit jako přípustná a stabilní heuristika pro A*.
2. Vytvoříme **RootNode** - pro každého agenta spustíme A* a získáme nejkratší možnou cestu. Zatím neuvažujeme ostatní agenty. Tyto cesty společně s prázdnou množinou Constraintů uložíme do RootNode a vložíme do minimové prioritní fronty, kde jako klíč bereme sumu přes délky cest.
3. Hlavní smyčka
 - (a) Z minimové haldy vyjmeme minimální vrchol a najdeme **konflikt** mezi jakoukoliv dvojicí agentů (A a B). Pokud neexistuje, našli jsme optimální řešení.
 - (b) Vytvoříme dva potomky **NodeA** a **NodeB**, které budou kopiemi vyjatého vrcholu. U **NodeA** přidáme do množiny Constraintů agenta A **Constraint** zabráňující nalezenému konfliktu a spustíme pro něj znovu A*. Symetricky pro agenta B. Tyto dva nové Nody vložíme do fronty a opakujeme.

Experiment

Byl použit benchmark sestávající ze 6 map z movingai sady (<https://movingai.com/benchmarks/mapf/index.html>):

- **random-32-32-10**, **random-64-64-10** – 10 % náhodných překážek,
- **room-32-32-4**, **room-64-64-8** – mřížka rozdělená na malé místnosti s úzkými dveřmi,
- **maze-32-32-2**, **maze-128-128-10** – bludiště s úzkými chodbami.

Každá mapa byla otestována s počtem agentů $k \in \{2, 4, 6, 8, 10, 12, 15, 20, 25, 30\}$ a 10 různými seedy, celkem tedy $6 \times 10 \times 10 = 600$ instancí. Pro každý (mapa, k , seed) scénář byly náhodně vygenerovány počáteční a cílové pozice tak, aby žádné dvě nekolidovaly.

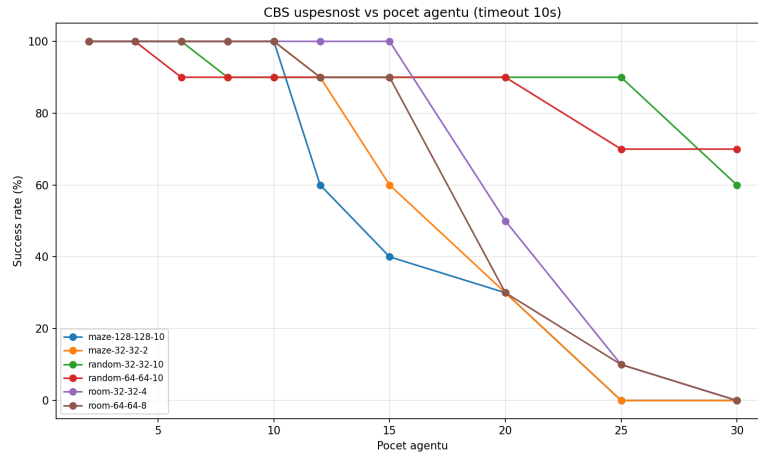
Časový limit jednoho běhu byl nastaven na **10 sekund**. Při překročení byla instance označena jako **timeout**.

Sledované metriky:

- **Success rate**: podíl instancí vyřešených pod limit.
- **Medián doby řešení** (jen úspěšné běhy – timeoutované by zkreslily průměr).
- **Medián počtu CBS uzlů**: strojově nezávislá metrika velikosti prohledávacího stromu.

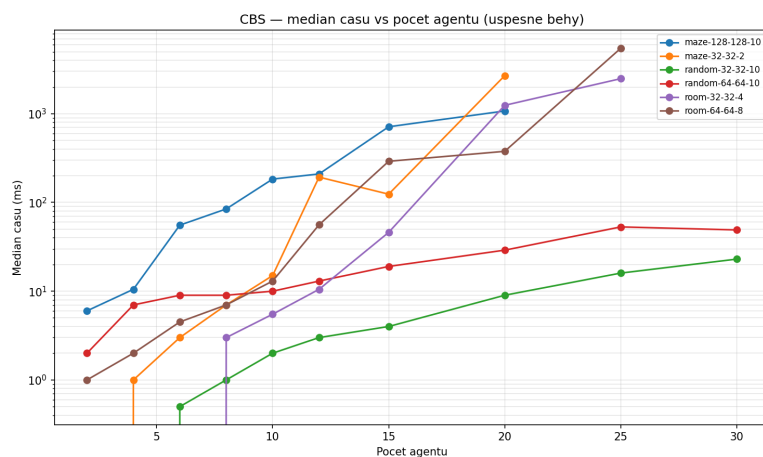
Výsledky

Úspěšnost dle počtu agentů



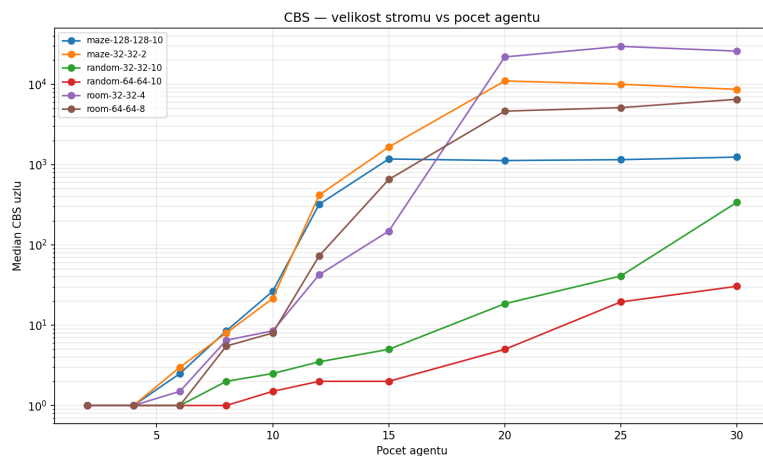
Obrázek 1: Success rate v závislosti na počtu agentů, podle typů map.

Medián času (logaritmická škála)



Obrázek 2: Medián doby řešení (jen úspěšné běhy). Logaritmická osa Y zviditelňuje exponenciální růst.

Velikost CBS stromu



Obrázek 3: Medián počtu expandovaných CBS uzlů.

Souhrnné tabulky pro vybrané mapy

k	success [%]	medián času [ms]	medián CBS uzlů	timeout
2	100	0	1	0
4	100	0	1	0
6	100	1	1	0
8	90	1	1	1
10	90	2	2	1
12	90	3	3	1
15	90	4	4	1
20	90	9	16	1
25	90	16	40	1
30	60	23	42	4

Tabulka 1: Detailní výsledky pro **random-32-32-10**.

k	success [%]	medián času [ms]	medián CBS uzlů	timeout
2	100	0	1	0
4	100	1	1	0
6	100	3	3	0
8	100	7	8	0
10	100	15	22	0
12	90	192	259	1
15	60	145	222	4
20	40	2686	2281	6
25	0	–	–	10
30	0	–	–	10

Tabulka 2: Detailní výsledky pro **maze-32-32-2** – bludiště s úzkými chodbami.

k	success [%]	medián času [ms]	medián CBS uzlů	timeout
2	100	2	1	0
4	100	7	1	0
6	90	9	1	1
8	90	9	1	1
10	90	10	1	1
12	90	13	2	1
15	90	19	2	1
20	90	29	4	1
25	70	53	10	3
30	70	49	22	3

Tabulka 3: Detailní výsledky pro **random-64-64-10**.

k	success [%]	medián času [ms]	medián CBS uzlů	timeout
2	100	0	1	0
4	100	0	1	0
6	100	0	2	0
8	100	3	7	0
10	100	6	9	0
12	100	11	43	0
15	100	46	149	0
20	50	1244	4231	5
25	10	2485	8439	9
30	0	–	–	10

Tabulka 4: Detailní výsledky pro **room-32-32-4** – mřížka rozdělená na malé místnosti s úzkými dveřmi.

Vliv výběru konfliktu

V CBS existuje volnost při výběru konfliktu k rozdělení uzlu. Testoval jsem dvě varianty: (a) první nalezený konflikt v lexikografickém pořadí agentů, (b) konflikt s nejnižším časem výskytu. Na párovaných instancích ($n = 282$, kde obě varianty dobehly úspěšně) varianta s nejnižším časem vyhrála v čase na 63 % případů (177 vs 82, Wilcoxon $p < 0.0001$). Medián zrychlení byl $1.19\times$, ale průměr $2.44\times$, což indikuje, že na většině instancí je zlepšení mírné, ale na podmnožině obtížných instancí dramatické.

Dále nová varianta vyřešila pod limitem 11 instancí, které původní nezvládla, oproti 1 v opačném směru. To znamená posun praktického limitu řešitelnosti.

Analýza přes jednotlivé mapy ukazuje, že efekt je největší na otevřených mapách (**random-64-64-10**: $1.38\times$) a nejmenší na bottleneck mapách (**room-64-64-8**: $1.02\times$).

Vyhodnocení

Z výsledků experimentu, jde jasně vidět, že daleko více než na velikosti mapy, záleží na její topologii. **Random 64x64** například zvládá i 30 agentů v řádu desítek milisekund, kdežto **Maze 32x32** začíná timeoutovat už při 12+ agentech. U mapy **Random 32x32**, která je menší, vidíme více timeoutů až u 30 agentů, což je očekávané chování. Dále můžeme pozorovat prudký pokles úspěšnosti u mapy **Room 32x32**, kde je počet otvorů mezi místnostmi omezený a při větším počtu agentů vzniká velký počet konfliktů, které nelze vyřešit bez exponenciálního prohledávání. Z grafu počtu CBS uzlů a mediánu času vidíme taky, že tyto dvě hodnoty koreluje.

Závěr

CBS algoritmus funguje a je optimální. Praktický limit pro tuto implementaci je maximálně 25-30 agentů na jednodušších/otevřenějších mapách. Na mapách s úzkými průchody úspěšnost prudce klesá a nelze za běhu řešit v rozumném čase pro více než 12-15 agentů. Jelikož HighLevel prohledávání CBS stromu přímo koreluje s mediánem času, nabízí se do budoucna optimalizace hledání konfliktů a taky omezení kopírování množiny Constraintů uvnitř CBS uzlů, což má potenciál snížit hlavní bottleneck algoritmu.